# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12**

---

**COURSE OUTCOME COVERED**

**CO1**: Components of a computer system, including CPU, memory, and I/O. Basic Operational Concepts: Instruction execution cycle, instruction fetch and decode.Bus Structures: Single-bus, multi-bus, and hierarchical buses.Factors of Performance: CPU execution time, CPI, clock cycles, and benchmarking.8085 and 8086 Architectures: Overview, instruction sets, and addressing modes. Modern Architectures: ARM Cortex, Intel Core, and AMD Ryzen.Instruction Sets, Formats, and Addressing Modes. Number Systems: Binary, hexadecimal, and their applications in computer systems. (BL4)
Assessment Tool Weightage: 40%

---

**QUESTIONS:5** **Total Marks:25**

**ANNEXURE A**
**APPLICATION BASED QUESTIONS**

***Code of Conduct:***

*I* VINIL KUMAR S (192511226) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

## ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.
4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

**MARKS ALLOCATION**

| Criteria | Understanding of Problem Context (20%) | Application of Concepts (20%) | Problem-Solving Approach (20%) | Accuracy of Solution (20%) | Clarity (20%) |
|---|---|---|---|---|---|
| **Q1** | | | | | |
| **Q2** | | | | | |
| **Q3** | | | | | |
| **Q4** | | | | | |
| **Q5** | | | | | |

## RUBRICS FOR CALCULATION:

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Problem Context | 20 | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| Application of Concepts | 30 | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts |
| Problem-Solving Approach | 20 | Logical, well-structured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach |
| Accuracy of Solution | 20 | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution |
| Clarity | 10 | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

# 1) PU, Memory and I/O Interaction in Smart City Surveillance

## Introduction

A smart city surveillance system processes continuous video from multiple cameras. During peak hours, the large amount of data transferred between **I/O devices, memory, and CPU** causes delays in threat detection.

## Interaction of CPU, Memory and I/O

**I/O Devices:** Cameras continuously send video data.

**Memory (RAM):** Stores video frames temporarily before processing.

**CPU:** Processes the data and detects threats.

When many cameras send data simultaneously, memory and CPU become overloaded. The CPU waits for data from memory, causing lag and slower threat detection.

## Single-Bus vs Multi-Bus Structure

| Single Bus | Multi Bus |
|---|---|
| CPU, memory and I/O share one bus. | Separate buses for CPU, memory and I/O. |
| High bus congestion. | Low congestion. |
| CPU waits for data transfer. | Parallel data transfer. |
| Slower performance. | Faster and more efficient performance. |

**Result:** A multi-bus structure improves speed and supports real-time video processing.

## Improving Instruction Execution Cycle

**Pipelining:** Executes multiple instructions simultaneously.

**Cache Memory:** Reduces memory access time.

**DMA (Direct Memory Access):** Transfers data directly between I/O and memory without CPU involvement.

**Multi-core Processing:** Processes multiple camera feeds at the same time.

These improvements reduce CPU waiting time and increase system performance.

### Conclusion

The lag occurs due to heavy data transfer between CPU, memory and I/O devices. Using a **multi-bus architecture** along with **pipelining, cache memory, DMA, and multi-core processing** reduces delays, improves throughput, and enables faster real-time threat detection.

## 2) Fetch–Decode–Execute Cycle in a Real-Time Stock Trading Application

### Introduction

A real-time stock trading application executes millions of instructions every second. During high load, frequent memory access and branch instructions increase the **Cycles Per Instruction (CPI)**, resulting in slower transaction processing.

### Impact of Fetch–Decode–Execute Cycle

**Fetch:** CPU fetches instructions from memory. Frequent memory access increases fetch time.

**Decode:** CPU interprets the instruction. Branch instructions may interrupt the instruction flow.

**Execute:** CPU performs calculations or data processing. Delays occur when the CPU waits for data from memory.

As CPI increases, the CPU takes more clock cycles to execute each instruction, leading to higher execution time and delayed transactions.

### Methods to Reduce CPI

**Pipelining:** Executes multiple instructions simultaneously, improving throughput.

**Cache Memory:** Stores frequently used data and instructions, reducing memory access time.

**Branch Prediction:** Predicts branch outcomes to avoid pipeline stalls.

**Out-of-Order Execution:** Executes independent instructions without waiting for previous ones.

**Multi-Core Processors:** Distributes transactions across multiple CPU cores for parallel processing.

## Conclusion

The fetch–decode–execute cycle directly affects execution time. High CPI caused by memory access and branch instructions slows transaction processing. Using **pipelining, cache memory, branch prediction, out-of-order execution, and multi-core processors** reduces CPI and improves CPU performance, enabling faster and more efficient real-time stock trading.

# 3) Addressing Modes in 8085 Microprocessor

## Introduction

An 8085-based industrial temperature control system receives data from sensors and controls actuators. Improper use of addressing modes may cause incorrect temperature readings, affecting system performance.

## Influence of Addressing Modes

**Immediate Addressing:** Data is provided directly in the instruction. Suitable for fixed values.

**Register Addressing:** Data is stored in CPU registers for fast access.

**Direct Addressing:** The memory address is specified directly in the instruction.

**Indirect Addressing:** The memory address is stored in a register pair (HL). Useful for accessing sensor data stored in memory.

**Implied Addressing:** The operand is implied by the instruction (e.g., CMA, RAL).

## Possible Causes of Errors

Incorrect memory address used in direct or indirect addressing.

Wrong register or register pair selected.

Improper data transfer between memory and registers.

Programming mistakes in sensor data processing.

Invalid instruction sequence causing incorrect temperature values.

## Using the 8085 Instruction Set Effectively

Use **LDA** and **STA** for accurate memory access.

Use **MOV** and **MVI** for correct data transfer.

Use **IN** and **OUT** instructions for sensor and actuator communication.

Use **CMP** and conditional jump instructions (**JZ, JNZ, JC**) for temperature comparison and decision making.

Use **HL register pair** carefully for indirect addressing to avoid accessing incorrect memory locations.

## Conclusion

Proper use of **addressing modes** and the **8085 instruction set** ensures accurate sensor data access and reliable actuator control. Careful programming reduces errors, improves temperature measurement accuracy, and enhances the overall reliability of the industrial temperature control system.

# 4) Segmentation in 8086 Microprocessor

## Introduction

The 8086 microprocessor uses **segmented memory** to organize programs into **Code, Data, Stack, and Extra segments**. Proper segment management ensures correct data access and smooth execution of multiple programs.

## Working of Segmentation

**Code Segment (CS):** Stores program instructions for execution.

**Data Segment (DS):** Stores variables and data used by the program.

**Stack Segment (SS):** Stores return addresses, local variables, and temporary data.

**Extra Segment (ES):** Used for additional data storage.

Each segment is accessed using its corresponding segment register along with an offset address.

## Problems Due to Improper Segment Handling

Incorrect segment register values lead to **wrong data access**.

Improper stack management causes **stack overflow or underflow**.

Incorrect use of segment registers may execute wrong instructions.

Invalid memory access can cause program crashes or unexpected results.

## Managing Instruction Execution and Addressing Modes

Load the correct **CS, DS, and SS** registers before execution.

Use proper **addressing modes** (Immediate, Register, Direct, Indirect, Based, Indexed) for correct memory access.

Balance **PUSH** and **POP** instructions to avoid stack overflow.

Update segment registers carefully during program switching.

Validate memory addresses before accessing data.

## Conclusion

Proper management of **code, data, and stack segments**, along with correct use of **instruction execution and addressing modes**, prevents incorrect data access and stack overflow. This ensures reliable and efficient execution of multiple programs in the **8086 microprocessor**.

# 5) Number System Conversion in Data Communication

## Introduction

In data communication systems, packet information is often represented in **hexadecimal** for easy reading and debugging. Since the CPU processes data in **binary**, accurate conversion between hexadecimal and binary is essential.

## Importance of Hexadecimal to Binary Conversion

Hexadecimal provides a compact representation of binary data.

The CPU executes instructions only in binary format.

Correct conversion ensures accurate packet processing and instruction execution.

It simplifies debugging and reduces data representation errors.

## Effects of Conversion Errors

Incorrect packet data is processed.

CPU may execute wrong instructions.

Data corruption and communication failures may occur.

Real-time system performance and reliability are affected.

## CPU Design and Benchmarking

**Efficient CPU Design:** Uses cache memory, pipelining, and error-checking mechanisms to improve performance.

**Benchmarking:** Measures CPU speed, accuracy, and execution time to identify conversion and processing errors.

Regular performance testing helps detect and minimize errors in real-time systems.

## Conclusion

Accurate **hexadecimal-to-binary conversion** is essential for correct instruction execution in data communication systems. Efficient CPU design and benchmarking improve processing speed, detect conversion errors early, and ensure reliable real-time system performance.